Assignment 12

Ising Model

Submitted by

Sanjeev Kumawat
`ph25mscst11026`

Instructor
Anupam Gupta

Date of Submission

April 23, 2026

Contents

1 Introduction **3**

2 Theory **3**

3 Methodology **3**

4 Results **4**

4.1 (a) Spin Configurations 4

4.2 (b) Magnetization vs Temperature 5

4.3 (c) Energy Fluctuation 5

5 Discussion **6**

6 Conclusion **6**

A Python Code **6**

Abstract

In this project, I simulated the two-dimensional Ising model using the Metropolis algorithm. The goal was to study how magnetization and energy fluctuations vary with temperature for a 50×50 lattice. The system was allowed to reach equilibrium before taking measurements to ensure accurate results. From the simulations, I observed a clear phase transition near the critical temperature $T \approx 2.27$. At low temperatures, the spins are ordered, while at high temperatures they become random. The results agree well with theoretical expectations.

1 Introduction

The Ising model is one of the simplest models used to study ferromagnetism in statistical physics. In this model, each site on a lattice contains a spin that can take values +1 or 1. These spins interact with their nearest neighbors, and the system evolves depending on temperature.
At low temperatures, spins tend to align with each other, resulting in an ordered (ferromagnetic) state. At high temperatures, thermal fluctuations dominate, and the spins become randomly oriented, leading to a disordered (paramagnetic) state. The transition between these two phases occurs at a specific temperature known as the critical temperature.
In this project, I used numerical simulation to study this phase transition and analyze how physical quantities like magnetization and energy fluctuations behave with temperature.

2 Theory

The energy of the system is given by the Ising Hamiltonian:

$$E = -J \sum_{\langle i,j \rangle} s_i s_j$$

where J is the interaction strength and the sum is taken over nearest neighbors. Each spin s_i can take values ± 1.
The magnetization of the system is defined as:

$$M = \frac{1}{N} \sum_i s_i$$

Magnetization serves as an order parameter. At low temperatures, it has a finite value, while at high temperatures it approaches zero.
To simulate the system, the Metropolis algorithm is used. In this method, a random spin is selected and flipped based on the change in energy. If the energy decreases, the flip is accepted. If the energy increases, the flip is accepted with probability:

$$P = e^{-\beta \Delta E}$$

where $\beta = \frac{1}{T}$.
The exact critical temperature for the two-dimensional Ising model is:

$$T_c \approx 2.269$$

3 Methodology

A two-dimensional square lattice of size 50×50 with periodic boundary conditions was considered. The interaction strength was taken as $J = 1$.
The simulation was performed using the Metropolis Monte Carlo method. Each Monte Carlo step consisted of L^2 spin updates, ensuring that all spins had a chance to flip.
To obtain reliable results, the simulation was divided into two phases:

- **Equilibration phase:** The system was allowed to evolve for 2000 steps to reach thermal equilibrium.

- **Measurement phase:** After equilibration, measurements were taken over 3000 steps.

To reduce statistical fluctuations, the results were averaged over multiple independent runs for each temperature. Parallel computation was used to accelerate the simulations.

4 Results

4.1 (a) Spin Configurations

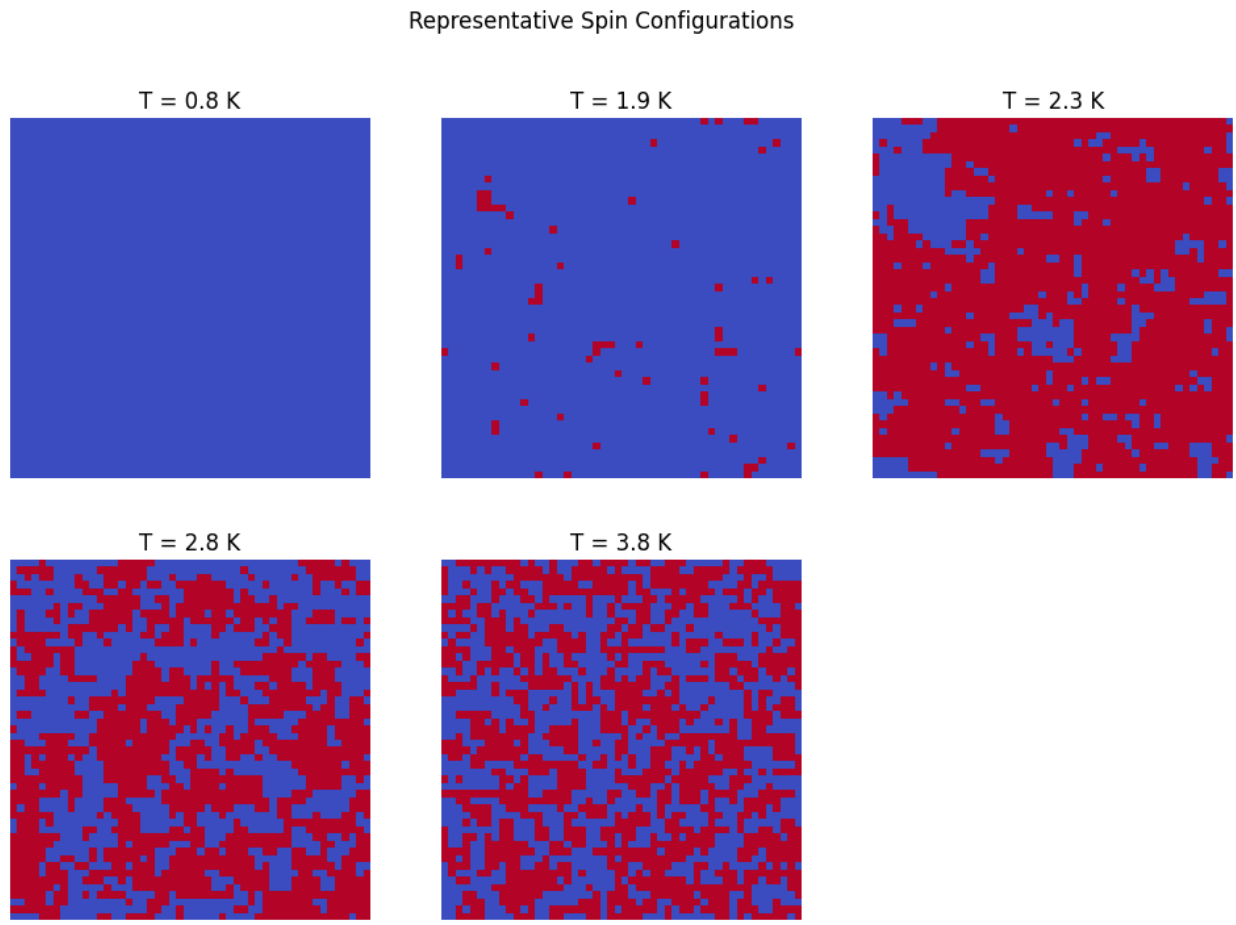

Figure 1

Spin configurations were plotted at five different temperatures: $T = 0.8, 1.9, 2.3, 2.8$, and 3.8. At low temperature ($T = 0.8$), almost all spins are aligned, indicating a ferromagnetic phase. As the temperature increases, clusters of opposite spins begin to form. Near the critical temperature ($T \approx 2.3$), large irregular clusters are observed, showing critical behavior. At higher temperatures ($T = 2.8$ and 3.8), the spins become randomly distributed, indicating a paramagnetic phase.

4.2 (b) Magnetization vs Temperature

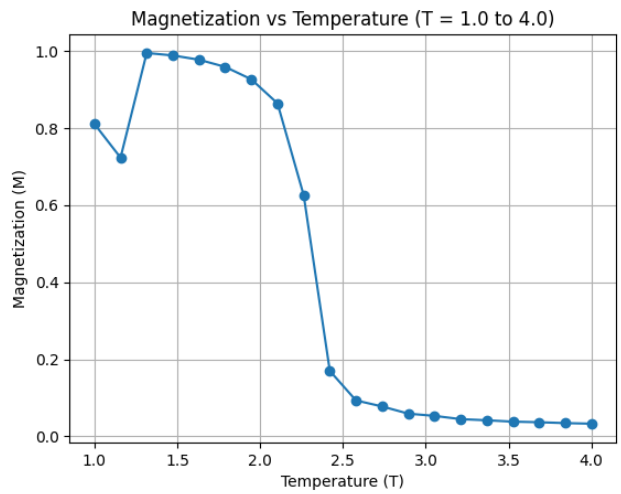

Figure 2

The magnetization was calculated as the average absolute value of spin over the lattice.
From the results, magnetization is close to 1 at low temperatures and decreases as temperature increases. Around $T \approx 2.2$–2.4, a sharp drop in magnetization is observed, indicating the phase transition. Beyond this region, magnetization approaches zero.

4.3 (c) Energy Fluctuation

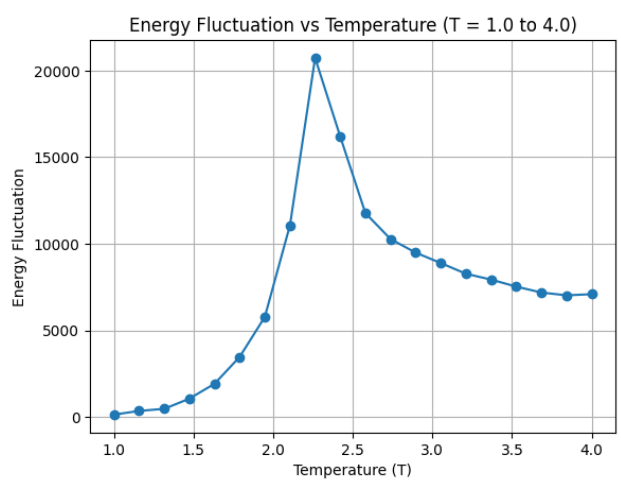

Figure 3

The results show that fluctuations are small at low temperatures, increase sharply near the critical temperature, and decrease again at higher temperatures. The peak in the fluctuation curve occurs near $T \approx 2.3$, which corresponds to the critical point.

5 Discussion

The simulation clearly demonstrates the phase transition from an ordered to a disordered state. At low temperatures, the system minimizes its energy by aligning spins, resulting in high magnetization. As the temperature increases, thermal fluctuations disrupt this order.
Near the critical temperature, the system becomes highly unstable, and large spin clusters form. This leads to maximum energy fluctuations. The observed critical temperature is close to the theoretical value of $T_c \approx 2.27$.
Since the lattice size is finite (50×50), the transition appears smooth rather than perfectly sharp. This behavior is expected due to finite-size effects.

6 Conclusion

In this project, the two-dimensional Ising model was successfully simulated using the Metropolis algorithm. The results show a clear phase transition from ferromagnetic to paramagnetic behavior. The critical temperature obtained from the simulation is close to the theoretical value. The study demonstrates how microscopic spin interactions give rise to macroscopic phenomena such as magnetism.

A Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
from multiprocessing import Pool

#Set seed
np.random.seed(1)

def initialize_spins(Lx, Ly):
    '''Create a 2d array of spin'''
    return np.random.choice([1, -1], size=(Lx, Ly))

def calculate_energy(spins, J):
    '''Calculation of Energy of the 2d lattice using periodic boundary
   condition'''
    kernel = np.array([
        [0, 1, 0],
        [1, 0, 1],
        [0, 1, 0]
    ])
    return -J * np.sum(spins * convolve2d(spins, kernel, mode="same",
   boundary="wrap")) / 2

def metropolis_step(spins, B, J):
    '''Perform single metropolis update on 2d lattice at random location''
   '
    Lx, Ly = spins.shape

    for _ in range(Lx * Ly):
```

```
        i = np.random.randint(Lx)
        j = np.random.randint(Ly)

        delta_E = 2 * J * spins[i, j] * (
            spins[(i-1)%Lx, j] + spins[(i+1)%Lx, j] +
            spins[i, (j+1)%Ly] + spins[i, (j-1)%Ly]
        )

        if delta_E <= 0 or np.random.rand() < np.exp(-B * delta_E):
            spins[i, j] *= -1

    return spins

def simulate_ising_2d(Lx, Ly, J, T, n_steps):
    """Run the 2D Ising model simulation."""
    B = 1.0 / T
    spins = initialize_spins(Lx, Ly)
    magnetizations = []
    energies = []

    for step in range(n_steps):
        spins = metropolis_step(spins, B, J)
        magnetizations.append(np.mean(spins))
        energies.append(calculate_energy(spins, J))

    return spins, magnetizations, energies


#Global parameters
J = 1.0
Lx, Ly = 50, 50
T_values = np.linspace(1, 4, 20)

equil_steps = 2000
measure_steps = 3000


#Parallel function for magnetization
def simulate_for_T(T):
    np.random.seed()   #different seed per process

    spins = initialize_spins(Lx, Ly)
    B = 1.0 / T

    mags = []

    #Equilibration
    for _ in range(equil_steps):
        spins = metropolis_step(spins, B, J)

    # Measurement
    for _ in range(measure_steps):
        spins = metropolis_step(spins, B, J)
        mags.append(np.mean(spins))
```

```
    return np.mean(np.abs(mags))


#Parallel function for energy fluctuation
def energy_for_T(T):
    np.random.seed()

    spins = initialize_spins(Lx, Ly)
    B = 1.0 / T

    energies = []

    for _ in range(equil_steps):
        spins = metropolis_step(spins, B, J)

    for _ in range(measure_steps):
        spins = metropolis_step(spins, B, J)
        energies.append(calculate_energy(spins, J))

    return np.var(energies)


#Representative spin states for the 5 different Temprature Values
def plot_representative_states():
    T_vals = [0.8, 1.9, 2.3, 2.8, 3.8]

    plt.figure(figsize=(12, 8))

    for i, T in enumerate(T_vals):
        spins, _, _ = simulate_ising_2d(Lx, Ly, J, T, 5000)

        plt.subplot(2, 3, i+1)
        plt.imshow(spins, cmap='coolwarm')
        plt.title(f"T = {T} K")
        plt.axis('off')

    plt.suptitle("Representative Spin Configurations")
    plt.show()


#Final execution
if __name__ == "__main__":

    #Parallel Magnetization
    with Pool() as pool:
        M_values = pool.map(simulate_for_T, T_values)

    plt.figure()
    plt.plot(T_values, M_values, 'o-')
    plt.xlabel("Temperature (T)")
    plt.ylabel("Magnetization (M)")
    plt.title(f"Magnetization vs Temperature (T = {T_values[0]:.1f} to {
   T_values[-1]:.1f})")
```

```
plt.grid()
plt.show()

#Parallel Energy Fluctuation
with Pool() as pool:
    F_values = pool.map(energy_for_T, T_values)

plt.figure()
plt.plot(T_values, F_values, 'o-')
plt.xlabel("Temperature (T)")
plt.ylabel("Energy Fluctuation")
plt.title(f"Energy Fluctuation vs Temperature (T = {T_values[0]:.1f}
to {T_values[-1]:.1f})")
plt.grid()
plt.show()

#Spin configurations
plot_representative_states()
```

References

[1] Sanjeev K, "Python code used in this report," GitHub repository, 2026. Available at: `https://github.com/arsanjeevk/Computational_Methods/tree/main/Assignment_12(Ising_Model)` [Accessed: 23 April 2026].